

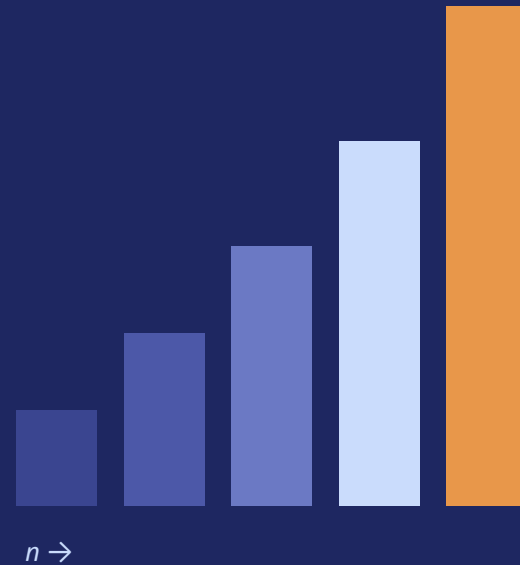
Understanding Growth Functions

A visual comparison of $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$

Course Code:CSA0621

Course Name: Design and Analysis of Algorithm

Presented by:V Deepa Dharshini



The Same Problem, Five Approaches

The problem: working with a contact list of 1,000,000 entries — the algorithm you pick decides whether it takes milliseconds or millennia

	$O(\log n)$	Binary search a sorted contact list for one name	<i>~20 comparisons</i>	Instant (< 1 μs)	Instant
	$O(n)$	Linear scan through every contact to find a match	<i>1,000,000 checks</i>	~1 millisecond	Fast
	$O(n \log n)$	Sort the entire list alphabetically (merge sort)	<i>~19,900,000 steps</i>	~20 milliseconds	Fast
	$O(n^2)$	Compare every contact against every other to find duplicates	<i>1,000,000,000,000 comparisons</i>	~16.7 minutes	Slow
	$O(2^n)$	Brute-force check every possible subset of contacts	<i>$2^{1,000,000}$ combinations</i>	Longer than the age of the universe	Infeasible

KEY INSIGHT

Same 1,000,000 contacts, same hardware — only the algorithm changes. Logarithmic and linear approaches finish instantly; quadratic takes minutes; exponential never finishes at all.

From Growth Rate to Algorithm Choice

Matching complexity class to real algorithms and typical use

$\log n$	$O(\log n)$	Binary search, balanced BST lookup	Ideal for huge datasets
n	$O(n)$	Linear scan, single-pass array processing	Scales well, predictable
$n \log n$	$O(n \log n)$	Merge sort, quicksort, heap sort	Best practical ceiling for sorting
n^2	$O(n^2)$	Bubble/insertion sort, nested-loop comparisons	Use only for small n
2^n	$O(2^n)$	Brute-force subsets, unoptimized recursion	Avoid at scale — redesign or approximate

Conclusion: Efficient algorithm selection means favoring logarithmic, linear, and log-linear growth whenever possible. Quadratic solutions should be reserved for small, bounded inputs, and exponential approaches should be redesigned — through better algorithms, heuristics, or approximation — before they reach production scale.